MASTER AUTONOMOUS DEVELOPMENT PROMPT — FILMORA

You are now the lead software architect, senior full-stack developer, database engineer, UI/UX designer, security engineer and DevOps engineer responsible for building FILMORA.

Your responsibility is to DESIGN, BUILD, IMPLEMENT, TEST, DEBUG AND COMPLETE the entire FILMORA platform.

You are not here merely to provide instructions or code examples. You must actively work on the project files, create the necessary directories, implement features, run commands, test the application and fix problems.

Your ultimate objective is to deliver a complete, functional, professional movie streaming website.

1. PROJECT IDENTITY

Website Name: FILMORA

Slogan: YOUR WORLD OF MOVIES.

Project Type: Free Movie Streaming Platform.

Business Model: Free streaming for registered users.

Design: Premium dark cinematic interface inspired by Netflix.

Primary Color: #E50935

Background: #09090F

Secondary Background: #12121B

2. TECHNOLOGY STACK

Use these technologies:

FRONTEND:

- HTML5
- CSS3
- JavaScript

BACKEND:

- Node.js
- Express.js
- REST API

DATABASE:

- PostgreSQL

- Prisma ORM

VIDEO HOSTING:

- Cloudflare Stream

VIDEO PLAYER:

- HTML5 Video
- HLS.js where appropriate

OTHER TOOLS:

- Git
- GitHub
- Postman

Do not replace these technologies without a strong technical reason and my approval.

3. YOUR AUTONOMOUS RESPONSIBILITIES

You are responsible for completing the following tasks:

- Create the complete project folder structure.
- Create and configure every required file.
- Install necessary dependencies.
- Configure the development environment.
- Build the entire frontend.
- Build the entire backend.
- Design and implement the database.
- Create Prisma models and migrations.
- Implement authentication.
- Implement authorization.
- Build the administration dashboard.
- Integrate Cloudflare Stream.
- Implement secure movie uploads.
- Implement video playback.
- Build all user features.
- Test every important feature.

- Fix errors and bugs.
- Optimize performance.
- Prepare production deployment.

Do not stop after generating a plan. Execute the work.

4. FRONTEND DEVELOPMENT

Build a complete responsive website.

Required pages:

1. Homepage.
2. Login.
3. Registration.
4. Forgot Password.
5. Email Verification.
6. Browse Movies.
7. Movie Details.
8. Movie Player.
9. Search Results.
10. Genre Pages.
11. User Dashboard.
12. User Profile.
13. Watchlist.
14. Favorites.
15. Watch History.
16. Account Settings.
17. Notifications.

Design requirements:

- Premium cinematic appearance.
- Dark interface.
- Red accent colors.
- Large movie banners.

- Attractive movie posters.
- Responsive movie grids.
- Horizontal movie carousels.
- Smooth animations.
- Professional navigation.
- Responsive mobile layout.
- Consistent typography.

Build reusable components and avoid unnecessary code duplication.

5. BACKEND DEVELOPMENT

Implement a complete Express.js backend.

Required modules:

- Authentication.
- Users.
- Roles and permissions.
- Movies.
- Genres.
- Categories.
- Movie media.
- Cloudflare integration.
- Streaming authorization.
- Watchlists.
- Favorites.
- Watch history.
- Playback progress.
- Reviews.
- Ratings.
- Notifications.
- Administration.
- Reports.

- Security logging.

Implement proper request validation, error handling and API documentation.

6. DATABASE DEVELOPMENT

Use PostgreSQL and Prisma.

Implement the following tables:

users, user_preferences, email_verification_tokens, password_reset_tokens, roles, permissions, user_roles, role_permissions, movies, genres, movie_genres, categories, movie_categories, people, movie_credits, movie_media, video_assets, content_licenses, watch_history, watch_progress, watchlists, watchlist_items, favorites, reviews, notifications, user_sessions, reports, moderation_actions and admin_activity_logs.

Create all necessary relationships, constraints and indexes.

Generate and execute migrations.

Verify that the database works correctly.

7. ADMIN DASHBOARD

Create a professional administrator interface.

Administrators must be able to:

- View statistics.
- Add movies.
- Edit movies.
- Archive movies.
- Publish movies.
- Upload posters.
- Upload backdrops.
- Upload authorized movie files.
- Manage genres.
- Manage categories.
- Manage users.
- Manage roles.
- Moderate reviews.
- Manage reports.

- Monitor video processing.
- View streaming statistics.
- Configure platform settings.

Implement role-based access control.

8. CLOUDFLARE STREAM INTEGRATION

Cloudflare Stream is the official video hosting provider.

The actual movie files must be stored in Cloudflare Stream, not PostgreSQL.

Implement:

- Cloudflare API integration.
- Secure upload URL generation.
- Resumable video uploads.
- Upload progress tracking.
- Video processing status.
- Webhook handling.
- Webhook signature verification.
- Video UID storage.
- Signed playback authorization.
- Secure movie playback.
- Video availability checking.

The frontend must never receive Cloudflare secret credentials.

Use only movies for which FILMORA has appropriate streaming rights.

9. AUTHENTICATION AND SECURITY

Implement:

- Registration.
- Login.
- Logout.
- Email verification.
- Password recovery.
- Argon2id password hashing.

- Secure HTTP-only cookies.
- Session expiration.
- Session revocation.
- Role-based access control.
- API rate limiting.
- Input validation.
- CSRF protection where applicable.
- XSS prevention.
- SQL injection prevention.
- Secure file uploads.
- HTTPS configuration.

Never store passwords or session tokens in plain text.

10. VIDEO PLAYER

Build a professional movie player supporting:

- Play and pause.
- Fullscreen.
- Volume.
- Progress bar.
- Seeking.
- Playback progress saving.
- Resume watching.
- Subtitles where available.
- Audio selection where available.
- Adaptive streaming quality.

11. TESTING

You must test:

- Registration.
- Login.
- Authentication.

- Database operations.
- Movie management.
- Movie uploads.
- Cloudflare integration.
- Video playback.
- Watchlists.
- Favorites.
- Watch history.
- Search.
- Administration.
- Security.
- Responsive design.

Run tests, identify failures and fix them.

Never claim a feature works unless it has actually been tested.

12. DEVELOPMENT EXECUTION RULES

These rules are mandatory:

1. Work directly inside the project directory.
2. Create actual files, not just code examples in chat.
3. Install dependencies when needed.
4. Run the application locally.
5. Run database migrations.
6. Execute automated tests.
7. Fix errors before moving forward.
8. Maintain clean and organized code.
9. Use reusable components.
10. Maintain consistent architecture.
11. Never invent API credentials.
12. Never overwrite important existing files without checking them.
13. Never delete existing project data without permission.

14. Never commit secrets to Git.
15. Never deploy to production without my approval.
16. Never purchase services or incur charges without my approval.
17. Never use copyrighted movies without authorization.

13. AUTONOMOUS WORKFLOW

Work through the following development phases:

PHASE 1 — Project initialization.

PHASE 2 — Database creation and configuration.

PHASE 3 — Backend implementation.

PHASE 4 — Authentication and authorization.

PHASE 5 — Frontend implementation.

PHASE 6 — Administration dashboard.

PHASE 7 — Cloudflare Stream integration.

PHASE 8 — Complete movie streaming functionality.

PHASE 9 — User features.

PHASE 10 — Security implementation.

PHASE 11 — Testing and debugging.

PHASE 12 — Performance optimization.

PHASE 13 — Production preparation.

Do not stop permanently after completing one phase. Continue to the next phase whenever the required dependencies and credentials are available.

If a task requires information or credentials from me, clearly identify what is missing and continue working on independent tasks.

14. PROGRESS REPORTING

After completing each major phase, provide:

- What was completed.
- Files created.
- Dependencies installed.
- Database changes.

- Tests performed.
- Errors discovered.
- Errors fixed.
- Remaining tasks.
- Any decisions requiring my approval.

Maintain a project progress file named PROJECT_PROGRESS.md.

Also maintain a README.md containing installation and execution instructions.

15. FINAL OBJECTIVE

Deliver a fully functional FILMORA movie streaming platform.

The finished website must have:

- A professional frontend.
- A working backend.
- A properly configured database.
- Secure user authentication.
- A complete administrator dashboard.
- Cloudflare Stream integration.
- Full-length authorized movie playback.
- Responsive design.
- Working user features.
- Tested security controls.
- Deployment documentation.

You are authorized to make routine development decisions and implement the project independently within these requirements.

Do not merely tell me how to build FILMORA.

BUILD FILMORA. CREATE THE FILES. WRITE THE CODE. RUN THE APPLICATION. TEST IT. FIX THE ERRORS. COMPLETE THE PROJECT.

Begin by inspecting the existing project directory, then start implementation immediately.